Real-time industry-style problem statement that naturally covers:

✅ Runtime Polymorphism
✅ Upcasting
✅ Downcasting
✅ Object Polymorphism
✅ Method Overriding
✅ instanceof Operator
✅ Dynamic Method Dispatch

Real-Time Problem Statement

Smart Vehicle Rental Management System

Background

A company named **DriveEasy Rentals** provides different types of vehicles on rent.

The company rents:

- Cars
- Bikes
- Electric Vehicles (EVs)

When a customer books a vehicle, the system should process all vehicles through a common interface, regardless of their actual type.

However, some vehicles have special features:

- Car → Air Conditioning
- Bike → Helmet Facility
- Electric Vehicle → Battery Health Monitoring

The management wants a flexible system where new vehicle types can be added in the future without changing existing code.

Requirements

1. Create a Parent Class

Create a class:

Vehicle

Common properties:

vehicleId
brand
rentPerDay

Common methods:

startVehicle()
calculateRent(int days)
showDetails()

---

2. Create Child Classes

Car

Additional Property:

hasSunroof

Override:

startVehicle()
showDetails()

Special Method:

enableAC()

Bike

Additional Property:

helmetIncluded

Override:

startVehicle()
showDetails()

Special Method:

provideHelmet()

ElectricVehicle

Additional Property:

batteryPercentage

Override:

startVehicle()
showDetails()

Special Method:

checkBatteryHealth()

Task 1: Demonstrate Runtime Polymorphism

Store different child objects inside parent references.

Example:

```
Vehicle v1 = new Car();
Vehicle v2 = new Bike();
Vehicle v3 = new ElectricVehicle();
```

Call:

```
v1.startVehicle();
v2.startVehicle();
v3.startVehicle();
```

Expected:

Actual overridden methods should execute according to object type.

This demonstrates:

Runtime Polymorphism

(Object decides method execution at runtime)

Task 2: Demonstrate Object Polymorphism

Create an array:

Vehicle vehicles[];

Store multiple types:

Car
Bike
ElectricVehicle

Loop through all vehicles and call:

showDetails()
calculateRent()

Without knowing actual object types.

This demonstrates:

Object Polymorphism

One reference type handling multiple object types.

Task 3: Demonstrate Upcasting

Store child object into parent reference.

Example:

Vehicle vehicle = new Car();

Questions for Students:

- Why is this allowed?
- What methods can be accessed?
- Which method executes?

This demonstrates:

Upcasting

Child → Parent

Task 4: Demonstrate Downcasting

Management wants to access AC feature only for cars.

Given:

Vehicle vehicle = new Car();

Convert:

Car car = (Car) vehicle;

Call:

car.enableAC();

This demonstrates:

Downcasting

Parent Reference → Child Reference

Task 5: Safe Downcasting

Before downcasting:

```
if(vehicle instanceof Car)
{
   Car car = (Car) vehicle;
   car.enableAC();
}
```

Students must explain:

- Why instanceof is needed?
- What exception can occur without it?

Expected answer:

ClassCastException

Business Scenario Extension

The company generates a daily report.

For every vehicle:

Vehicle vehicle

System should:

```
vehicle.showDetails();
vehicle.startVehicle();
```

And additionally:

- Enable AC if Car
- Provide Helmet if Bike
- Check Battery if EV

Using:

instanceof

and

Downcasting

Expected Learning Outcomes

After completing this project students will understand:

Concept	Covered
Inheritance	✅
Method Overriding	✅
Runtime Polymorphism	✅
Dynamic Method Dispatch	✅
Object Polymorphism	✅
Upcasting	✅
Downcasting	✅
instanceof	✅
ClassCastException	✅
Real World OOP Design	✅

Challenge Task (Advanced)

Add a new vehicle type:

LuxuryCar

Features:

- Chauffeur Service
- Premium Insurance

Without modifying existing rental processing code.

Students should prove that **Polymorphism** makes the system easily extensible and follows the **Open-Closed Principle (OCP)**. This is how OOP is used in real rental, banking, insurance, and enterprise applications.

Real-Time Problem Statement

Smart Payment Gateway System

Background

A fintech company is developing an online payment platform similar to payment systems used by e-commerce websites.

Customers can pay using:

- Credit Card
- UPI
- Net Banking
- Digital Wallet

The payment processing system should work with all payment methods through a common reference without knowing the actual payment type.

However, each payment method has its own special features.

Business Requirement

Whenever a customer makes a payment:

1. Validate the payment.
2. Process the transaction.
3. Generate a receipt.
4. Apply method-specific logic.

The system should be easily extendable for future payment methods.

Class Design

Parent Class

Payment

Properties:

paymentId
amount
customerName

Methods:

processPayment()
generateReceipt()
showPaymentDetails()

Child Class 1

CreditCardPayment

Additional Property:

cardNumber

Override:

processPayment()
showPaymentDetails()

Special Method:

checkCreditLimit()

Child Class 2

UPIPayment

Additional Property:

upiId

Override:

processPayment()

showPaymentDetails()

Special Method:

verifyUPIPin()

Child Class 3

WalletPayment

Additional Property:

walletBalance

Override:

processPayment()
showPaymentDetails()

Special Method:

checkWalletBalance()

Task 1: Runtime Polymorphism

Store different payment objects in parent references.

```
Payment p1 = new CreditCardPayment();
Payment p2 = new UPIPayment();
Payment p3 = new WalletPayment();
```

Call:

```
p1.processPayment();
p2.processPayment();
p3.processPayment();
```

Expected

Different overridden methods should execute based on the actual object.

This demonstrates:

✅ Runtime Polymorphism

Task 2: Object Polymorphism

Create an array:

Payment payments[];

Store:

CreditCardPayment
UPIPayment
WalletPayment

Process all payments:

```
for(Payment p : payments)
{
   p.processPayment();
   p.generateReceipt();
}
```

One reference type handling multiple objects.

This demonstrates:

✅ Object Polymorphism

Task 3: Upcasting

Store child object in parent reference.

Payment payment = new UPIPayment();

Questions:

1. Is this Upcasting?
2. Which methods are accessible?
3. Which method executes?

Expected Concept:

✅ Upcasting

Task 4: Downcasting

Fraud Detection Team wants access to UPI-specific functionality.

Given:

Payment payment = new UPIPayment();

Convert:

UPIPayment upi = (UPIPayment) payment;

Call:

upi.verifyUPIPin();

This demonstrates:

✅ Downcasting

Task 5: Safe Downcasting

Before casting:

```
if(payment instanceof UPIPayment)
{
   UPIPayment upi = (UPIPayment) payment;
   upi.verifyUPIPin();
}
```

Students must explain:

- Why instanceof is required?
- What happens if we cast a CreditCardPayment to UPIPayment?

Expected:

ClassCastException

Real Industry Scenario

At the end of the day, the company generates a transaction report.

All payments are stored as:

List<Payment>

System should:

payment.processPayment();
payment.generateReceipt();

Additionally:

- Check Credit Limit for Credit Card payments
- Verify UPI PIN for UPI payments
- Check Wallet Balance for Wallet payments

Using:

instanceof

and

Downcasting

Challenge Task

Add a new payment method:

CryptoPayment

Features:

validateWalletAddress()

Requirement:

The existing payment processing code should work without modification.

Students must explain:

- How Runtime Polymorphism helps?
- Why Upcasting is useful?
- Where Downcasting is required?
- How Object Polymorphism reduces code duplication?

Learning Outcomes

Concept	Covered
Inheritance	✅
Method Overriding	✅
Runtime Polymorphism	✅
Dynamic Method Dispatch	✅
Object Polymorphism	✅
Upcasting	✅
Downcasting	✅
instanceof	✅
ClassCastException	✅
Enterprise-Level Design	✅

This scenario closely resembles real payment gateways used by companies such as Razorpay, Paytm, and PhonePe.

Real-Time Problem Statement

Smart Employee Management System

Background

A multinational IT company manages different types of employees.

The company has:

- Full-Time Employees
- Part-Time Employees
- Contract Employees

All employees work for the same organization, but salary calculation and benefits differ based on employee type.

The HR department wants a flexible system where all employees can be managed through a common reference while allowing employee-specific operations.

Business Requirements

The system should:

1. Store employee information.
2. Calculate salary according to employee type.
3. Generate monthly payroll reports.
4. Allow HR to access employee-specific features when required.
5. Support future employee types without changing existing payroll logic.

Class Design

Parent Class

Employee

Properties:

employeeId
employeeName
department

Methods:

calculateSalary()
generatePayslip()
showEmployeeDetails()

Child Class 1

FullTimeEmployee

Additional Property:

monthlySalary

Override:

calculateSalary()
showEmployeeDetails()

Special Method:

applyAnnualBonus()

Child Class 2

PartTimeEmployee

Additional Property:

hoursWorked
hourlyRate

Override:

calculateSalary()
showEmployeeDetails()

Special Method:

calculateOvertime()

Child Class 3

ContractEmployee

Additional Property:

contractAmount

Override:

calculateSalary()
showEmployeeDetails()

Special Method:

extendContract()

Task 1: Runtime Polymorphism

Store different employee objects using parent references.

```
Employee e1 = new FullTimeEmployee();
Employee e2 = new PartTimeEmployee();
Employee e3 = new ContractEmployee();
```

Process payroll:

```
e1.calculateSalary();
e2.calculateSalary();
e3.calculateSalary();
```

Expected

Different salary calculation methods should execute automatically based on the actual object type.

Concept Covered

✅ Runtime Polymorphism

✅ Dynamic Method Dispatch

Task 2: Object Polymorphism

Create:

Employee employees[];

Store all employee types.

Loop through all employees:

```
for(Employee emp : employees)
{
   emp.showEmployeeDetails();
   emp.calculateSalary();
   emp.generatePayslip();
}
```

Expected

One reference type handles multiple employee objects.

Concept Covered

✅ Object Polymorphism

Task 3: Upcasting

Store child object into parent reference.

Employee emp = new FullTimeEmployee();

Questions for Students:

1. Is this Upcasting?
2. Why is it allowed?
3. Which methods are accessible?
4. Which version of calculateSalary() executes?

Concept Covered

✅ Upcasting

Task 4: Downcasting

HR wants to give an annual bonus only to Full-Time Employees.

Given:

Employee emp = new FullTimeEmployee();

Convert:

FullTimeEmployee ft = (FullTimeEmployee) emp;

Call:

ft.applyAnnualBonus();

Concept Covered

✅ Downcasting

Task 5: Safe Downcasting

Before casting:

```
if(emp instanceof FullTimeEmployee)
{
   FullTimeEmployee ft =
      (FullTimeEmployee) emp;
```

```
    ft.applyAnnualBonus();
}
```

Students must explain:

- Why instanceof is needed?
- What happens if PartTimeEmployee is cast to FullTimeEmployee?

Expected

ClassCastException

Real Industry Scenario

Every month, the Payroll Department receives a list:

List<Employee>

The system should:

```
calculateSalary();
generatePayslip();
showEmployeeDetails();
```

for every employee.

Additionally:

- Apply bonus for Full-Time Employees
- Calculate overtime for Part-Time Employees
- Extend contract for Contract Employees

Using:

instanceof

and

Downcasting

Challenge Task

The company introduces a new employee type:

InternEmployee

Special Feature:

assignMentor()

Requirement:

The payroll system should support InternEmployee without modifying existing payroll processing code.

Students must explain:

1. How Runtime Polymorphism helps?
2. Why Upcasting is used throughout the payroll system?
3. When Downcasting becomes necessary?
4. How Object Polymorphism makes the system scalable?

Why Students Like This Problem

This scenario is relatable because every student understands:

- Employees
- Salaries
- HR Department
- Payroll Processing

While solving it, they naturally use:

✅ Inheritance
✅ Method Overriding
✅ Runtime Polymorphism
✅ Object Polymorphism
✅ Upcasting
✅ Downcasting
✅ instanceof
✅ Dynamic Method Dispatch
✅ Real-world Enterprise Design

This is very similar to payroll systems used in companies like Tata Consultancy Services, Infosys, and Wipro.

Real-Time Problem Statement

Smart Employee Management System

Background

A multinational IT company manages different types of employees.

The company has:

- Full-Time Employees
- Part-Time Employees
- Contract Employees

All employees work for the same organization, but salary calculation and benefits differ based on employee type.

The HR department wants a flexible system where all employees can be managed through a common reference while allowing employee-specific operations.

Business Requirements

The system should:

1. Store employee information.
2. Calculate salary according to employee type.
3. Generate monthly payroll reports.
4. Allow HR to access employee-specific features when required.
5. Support future employee types without changing existing payroll logic.

Class Design

Parent Class

Employee

Properties:

employeeId
employeeName
department

Methods:

calculateSalary()
generatePayslip()
showEmployeeDetails()

Child Class 1

FullTimeEmployee

Additional Property:

monthlySalary

Override:

calculateSalary()
showEmployeeDetails()

Special Method:

applyAnnualBonus()

Child Class 2

PartTimeEmployee

Additional Property:

hoursWorked
hourlyRate

Override:

calculateSalary()
showEmployeeDetails()

Special Method:

calculateOvertime()

Child Class 3

ContractEmployee

Additional Property:

contractAmount

Override:

calculateSalary()
showEmployeeDetails()

Special Method:

extendContract()

Task 1: Runtime Polymorphism

Store different employee objects using parent references.

```
Employee e1 = new FullTimeEmployee();
Employee e2 = new PartTimeEmployee();
Employee e3 = new ContractEmployee();
```

Process payroll:

```
e1.calculateSalary();
e2.calculateSalary();
e3.calculateSalary();
```

Expected

Different salary calculation methods should execute automatically based on the actual object type.

Concept Covered

✅ Runtime Polymorphism

✅ Dynamic Method Dispatch

Task 2: Object Polymorphism

Create:

Employee employees[];

Store all employee types.

Loop through all employees:

```
for(Employee emp : employees)
{
   emp.showEmployeeDetails();
   emp.calculateSalary();
   emp.generatePayslip();
}
```

Expected

One reference type handles multiple employee objects.

Concept Covered

✅ Object Polymorphism

Task 3: Upcasting

Store child object into parent reference.

Employee emp = new FullTimeEmployee();

Questions for Students:

1. Is this Upcasting?
2. Why is it allowed?
3. Which methods are accessible?
4. Which version of calculateSalary() executes?

Concept Covered

✅ Upcasting

Task 4: Downcasting

HR wants to give an annual bonus only to Full-Time Employees.

Given:

Employee emp = new FullTimeEmployee();

Convert:

FullTimeEmployee ft = (FullTimeEmployee) emp;

Call:

ft.applyAnnualBonus();

Concept Covered

✅ Downcasting

Task 5: Safe Downcasting

Before casting:

```
if(emp instanceof FullTimeEmployee)
{
   FullTimeEmployee ft =
      (FullTimeEmployee) emp;
```

```
    ft.applyAnnualBonus();
}
```

Students must explain:

- Why instanceof is needed?
- What happens if PartTimeEmployee is cast to FullTimeEmployee?

Expected

ClassCastException

Real Industry Scenario

Every month, the Payroll Department receives a list:

List<Employee>

The system should:

```
calculateSalary();
generatePayslip();
showEmployeeDetails();
```

for every employee.

Additionally:

- Apply bonus for Full-Time Employees
- Calculate overtime for Part-Time Employees
- Extend contract for Contract Employees

Using:

instanceof

and

Downcasting

Challenge Task

The company introduces a new employee type:

InternEmployee

Special Feature:

assignMentor()

Requirement:

The payroll system should support InternEmployee without modifying existing payroll processing code.

Students must explain:

1. How Runtime Polymorphism helps?
2. Why Upcasting is used throughout the payroll system?
3. When Downcasting becomes necessary?
4. How Object Polymorphism makes the system scalable?

Why Students Like This Problem

This scenario is relatable because every student understands:

- Employees
- Salaries
- HR Department
- Payroll Processing

While solving it, they naturally use:

✅ Inheritance
✅ Method Overriding
✅ Runtime Polymorphism
✅ Object Polymorphism
✅ Upcasting
✅ Downcasting
✅ instanceof
✅ Dynamic Method Dispatch
✅ Real-world Enterprise Design

This is very similar to payroll systems used in companies like Tata Consultancy Services, Infosys, and Wipro.

Real-Time Problem Statement

Smart Employee Management System

Background

A multinational IT company manages different types of employees.

The company has:

- Full-Time Employees
- Part-Time Employees
- Contract Employees

All employees work for the same organization, but salary calculation and benefits differ based on employee type.

The HR department wants a flexible system where all employees can be managed through a common reference while allowing employee-specific operations.

Business Requirements

The system should:

1. Store employee information.
2. Calculate salary according to employee type.
3. Generate monthly payroll reports.
4. Allow HR to access employee-specific features when required.
5. Support future employee types without changing existing payroll logic.

Class Design

Parent Class

Employee

Properties:

employeeId
employeeName
department

Methods:

calculateSalary()
generatePayslip()
showEmployeeDetails()

Child Class 1

FullTimeEmployee

Additional Property:

monthlySalary

Override:

calculateSalary()
showEmployeeDetails()

Special Method:

applyAnnualBonus()

Child Class 2

PartTimeEmployee

Additional Property:

hoursWorked
hourlyRate

Override:

calculateSalary()
showEmployeeDetails()

Special Method:

calculateOvertime()

Child Class 3

ContractEmployee

Additional Property:

contractAmount

Override:

calculateSalary()
showEmployeeDetails()

Special Method:

extendContract()

Task 1: Runtime Polymorphism

Store different employee objects using parent references.

```
Employee e1 = new FullTimeEmployee();
Employee e2 = new PartTimeEmployee();
Employee e3 = new ContractEmployee();
```

Process payroll:

```
e1.calculateSalary();
e2.calculateSalary();
e3.calculateSalary();
```

Expected

Different salary calculation methods should execute automatically based on the actual object type.

Concept Covered

✅ Runtime Polymorphism

✅ Dynamic Method Dispatch

Task 2: Object Polymorphism

Create:

Employee employees[];

Store all employee types.

Loop through all employees:

```
for(Employee emp : employees)
{
   emp.showEmployeeDetails();
   emp.calculateSalary();
   emp.generatePayslip();
}
```

Expected

One reference type handles multiple employee objects.

Concept Covered

✅ Object Polymorphism

Task 3: Upcasting

Store child object into parent reference.

Employee emp = new FullTimeEmployee();

Questions for Students:

1. Is this Upcasting?
2. Why is it allowed?
3. Which methods are accessible?
4. Which version of calculateSalary() executes?

Concept Covered

✅ Upcasting

Task 4: Downcasting

HR wants to give an annual bonus only to Full-Time Employees.

Given:

Employee emp = new FullTimeEmployee();

Convert:

FullTimeEmployee ft = (FullTimeEmployee) emp;

Call:

ft.applyAnnualBonus();

Concept Covered

✅ Downcasting

Task 5: Safe Downcasting

Before casting:

```
if(emp instanceof FullTimeEmployee)
{
   FullTimeEmployee ft =
      (FullTimeEmployee) emp;
```

```
    ft.applyAnnualBonus();
}
```

Students must explain:

- Why instanceof is needed?
- What happens if PartTimeEmployee is cast to FullTimeEmployee?

Expected

ClassCastException

Real Industry Scenario

Every month, the Payroll Department receives a list:

List<Employee>

The system should:

```
calculateSalary();
generatePayslip();
showEmployeeDetails();
```

for every employee.

Additionally:

- Apply bonus for Full-Time Employees
- Calculate overtime for Part-Time Employees
- Extend contract for Contract Employees

Using:

instanceof

and

Downcasting

Challenge Task

The company introduces a new employee type:

InternEmployee

Special Feature:

assignMentor()

Requirement:

The payroll system should support InternEmployee without modifying existing payroll processing code.

Students must explain:

1. How Runtime Polymorphism helps?
2. Why Upcasting is used throughout the payroll system?
3. When Downcasting becomes necessary?
4. How Object Polymorphism makes the system scalable?

Why Students Like This Problem

This scenario is relatable because every student understands:

- Employees
- Salaries
- HR Department
- Payroll Processing

While solving it, they naturally use:

✅ Inheritance
✅ Method Overriding
✅ Runtime Polymorphism
✅ Object Polymorphism
✅ Upcasting
✅ Downcasting
✅ instanceof
✅ Dynamic Method Dispatch
✅ Real-world Enterprise Design

This is very similar to payroll systems used in companies like Tata Consultancy Services, Infosys, and Wipro.

Real-Time Problem Statement

Smart E-Learning Platform (Udemy / Coursera Style)

Background

An online learning platform wants to manage different types of courses available for students.

The platform offers:

- Recorded Courses
- Live Courses
- Corporate Training Programs

All courses can be enrolled in and launched, but each course type has its own learning experience and business rules.

The management wants a scalable design where future course types can be added without modifying existing platform code.

Business Requirements

The platform should:

1. Allow students to enroll in courses.
2. Launch course content.
3. Generate completion certificates.
4. Support course-specific features.
5. Manage all course types through a common interface.

Class Design

Parent Class

Course

Properties

courseId
courseTitle
courseFee

Methods

enroll()
startLearning()
generateCertificate()
showCourseDetails()

Child Class 1

RecordedCourse

Additional Property

videoDuration

Override Methods

startLearning()
showCourseDetails()

Special Method

downloadVideos()

Child Class 2

LiveCourse

Additional Property

liveSessionSchedule

Override Methods

startLearning()
showCourseDetails()

Special Method

joinLiveSession()

Child Class 3

CorporateTraining

Additional Property

companyName

Override Methods

startLearning()
showCourseDetails()

Special Method

generateCompanyReport()

Task 1: Runtime Polymorphism

Students enroll in different courses.

```
Course c1 = new RecordedCourse();
Course c2 = new LiveCourse();
Course c3 = new CorporateTraining();
```

Launch learning:

```
c1.startLearning();
c2.startLearning();
c3.startLearning();
```

Expected

Each course launches differently:

- RecordedCourse → Plays recorded videos
- LiveCourse → Opens live classroom
- CorporateTraining → Opens corporate learning portal

Concepts Covered

✅ Runtime Polymorphism
✅ Method Overriding
✅ Dynamic Method Dispatch

Task 2: Object Polymorphism

Store all courses in:

List<Course> courses

Process:

```
for(Course course : courses)
{
   course.enroll();
   course.startLearning();
   course.generateCertificate();
}
```

Expected

A single parent reference handles multiple child objects.

Concept Covered

✅ Object Polymorphism

Task 3: Upcasting

Store a child object in a parent reference.

Course course =

new LiveCourse();

Student Questions

1. Is this Upcasting?
2. Why is it allowed?
3. Which methods can be accessed?
4. Which version of startLearning() executes?

Concept Covered

✅ Upcasting

Task 4: Downcasting

Training coordinator wants to access live session functionality.

Given:

```
Course course =
    new LiveCourse();
```

Convert:

```
LiveCourse live =
    (LiveCourse) course;
```

Call:

```
live.joinLiveSession();
```

Concept Covered

✅ Downcasting

Task 5: Safe Downcasting

Before casting:

```
if(course instanceof LiveCourse)
{
   LiveCourse live =
     (LiveCourse) course;

   live.joinLiveSession();
}
```

Questions

1. Why is instanceof required?
2. What happens if RecordedCourse is cast to LiveCourse?

Expected

ClassCastException

Real Industry Scenario

An online learning company stores all purchased courses as:

List<Course>

Whenever a student logs in:

```
course.showCourseDetails();
course.startLearning();
```

are executed for every course.

Additional actions:

Recorded Course

downloadVideos();

Live Course

joinLiveSession();

Corporate Training

generateCompanyReport();

using:

instanceof

and

Downcasting

Challenge Task

The platform launches a new course type:

AICourse

Special Features:

evaluateCodingAssignment();
generateAIProgressReport();

Requirement

Existing course-processing code should work without any modification.

Students must explain:

- How Runtime Polymorphism helps add new course types.
- Why Upcasting is used throughout the platform.
- When Downcasting becomes necessary.
- How Object Polymorphism simplifies management of thousands of courses.

Extra Engagement Twist

Scenario

You are the Lead Java Developer of **LearnHub**, an e-learning platform.

The CEO says:

> "Next month we will launch 20 new course types. I don't want developers changing the main processing code every time."

Your Task

Design the system using:

- Inheritance
- Method Overriding
- Runtime Polymorphism
- Object Polymorphism
- Upcasting
- Downcasting
- instanceof

so that the platform can support unlimited future course types with minimal code changes.

Concepts Covered

Concept	Covered
Inheritance	✅
Method Overriding	✅
Runtime Polymorphism	✅
Dynamic Method Dispatch	✅
Object Polymorphism	✅
Upcasting	✅
Downcasting	✅
instanceof	✅
ClassCastException	✅
Collections	✅
Real Enterprise Architecture	✅

This problem is especially engaging for students because it relates directly to online learning platforms they use every day and mirrors how large-scale educational software is designed.

Real-Time Problem Statement

Smart Banking System – Loan Processing Platform

Background

A bank wants to automate its loan processing system.

The bank offers multiple types of loans:

- Home Loan
- Car Loan
- Education Loan

Every loan application follows a common process:

1. Apply for Loan
2. Verify Documents
3. Calculate Interest
4. Approve or Reject Loan

However, each loan type has its own eligibility rules, interest calculation, and benefits.

The bank wants a flexible design where new loan types can be introduced without changing the existing loan processing engine.

Your Role

You are a Java Developer in the bank's Loan Management Team.

Your task is to design a system using OOP concepts.

The solution must demonstrate:

✅ Runtime Polymorphism
✅ Object Polymorphism
✅ Upcasting
✅ Downcasting
✅ Method Overriding

✅ instanceof
✅ Dynamic Method Dispatch

Class Design

Parent Class

Loan

Common Properties

loanId
customerName
loanAmount

Common Methods

applyLoan()
verifyDocuments()
calculateInterest()
displayLoanDetails()

Child Class 1

HomeLoan

Additional Property

propertyValue

Override Methods

calculateInterest()
displayLoanDetails()

Special Method

verifyPropertyDocuments()

Child Class 2

CarLoan

Additional Property

vehicleModel

Override Methods

calculateInterest()
displayLoanDetails()

Special Method

verifyVehicleInsurance()

Child Class 3

EducationLoan

Additional Property

collegeName

Override Methods

calculateInterest()
displayLoanDetails()

Special Method

verifyAdmissionLetter()

Task 1: Runtime Polymorphism

The bank stores all loans using parent references.

Loan l1 = new HomeLoan();
Loan l2 = new CarLoan();

Loan l3 = new EducationLoan();

Now calculate interest:

```
l1.calculateInterest();
l2.calculateInterest();
l3.calculateInterest();
```

Expected

Different interest calculation logic should execute based on the actual object type.

Student Observation

Although the reference type is `Loan`, the overridden method of the actual object executes.

This demonstrates:

✅ Runtime Polymorphism
✅ Dynamic Method Dispatch

Task 2: Object Polymorphism

Store all loan applications in:

List<Loan> loans

Process them using one loop:

```
for(Loan loan : loans)
{
   loan.verifyDocuments();
   loan.calculateInterest();
   loan.displayLoanDetails();
}
```

Expected

One parent reference manages multiple child objects.

Concept Covered

✅ Object Polymorphism

Task 3: Upcasting

Loan officer stores a Home Loan object as:

Loan loan = new HomeLoan();

Student Questions

1. Is this Upcasting?
2. Why is it allowed?
3. Which methods are accessible?
4. Which calculateInterest() method executes?

Concept Covered

✅ Upcasting

Task 4: Downcasting

Property Verification Department wants access to HomeLoan-specific functionality.

Given:

Loan loan = new HomeLoan();

Convert:

```
HomeLoan homeLoan =
      (HomeLoan) loan;
```

Call:

homeLoan.verifyPropertyDocuments();

Concept Covered

✅ Downcasting

Task 5: Safe Downcasting

Before casting:

```
if(loan instanceof HomeLoan)
{
   HomeLoan homeLoan =
      (HomeLoan) loan;

   homeLoan.verifyPropertyDocuments();
}
```

Student Questions

1. Why is `instanceof` required?
2. What happens if a CarLoan object is cast to HomeLoan?

Expected

ClassCastException

Real Banking Scenario

At the end of the day, the bank receives 50,000 loan applications.

All applications are stored as:

List<Loan>

The processing engine should work like this:

```
loan.verifyDocuments();
loan.calculateInterest();
loan.displayLoanDetails();
```

without knowing the actual loan type.

Additional actions:

Home Loan

verifyPropertyDocuments();

Car Loan

verifyVehicleInsurance();

Education Loan

verifyAdmissionLetter();

using:

instanceof

and

Downcasting

Challenge Task (Highly Engaging)

New CEO Requirement

The bank launches a new loan product:

StartupBusinessLoan

Special Features:

verifyBusinessPlan();
evaluateStartupRisk();

Requirement

The existing loan processing engine must continue working without modification.

Final Discussion Questions

Ask students:

Q1

Why does the following code work?

```
Loan loan = new HomeLoan();
```

Q2

Why does the following code fail?

```
Loan loan = new CarLoan();

HomeLoan h =
    (HomeLoan) loan;
```

Q3

Why does the bank prefer:

```
List<Loan>
```

instead of:

```
List<HomeLoan>
List<CarLoan>
List<EducationLoan>
```

Q4

How does Runtime Polymorphism help when new loan types are introduced?

Q5

Where exactly are Upcasting and Downcasting used in real banking software?

Why Students Love This Scenario

Students can relate to:

- Home Loans
- Car Loans
- Education Loans
- Banking Apps
- EMI Calculations

The business logic feels real, and while implementing it they naturally practice:

✅ Inheritance
✅ Method Overriding
✅ Runtime Polymorphism
✅ Object Polymorphism
✅ Upcasting
✅ Downcasting
✅ instanceof
✅ ClassCastException
✅ Enterprise-Level Design Thinking

This type of design is very similar to systems used in banks such as State Bank of India, HDFC Bank, and ICICI Bank.